Practice Assignment: Containerizing a Flash-Sale Ticketing Platform

1. Scenario

You are the DevOps engineer at TicketRush, a startup that sells limited concert tickets in flash sales. Last month the team demoed the platform on a developer laptop and it fell over: the API crashed under load, tickets were oversold, and nobody could reproduce the setup on a second machine. The CTO has given you one week to containerize the platform so that any engineer can start it with a single command, and so that the same images can be shipped to production.

The code is already written by the developers. **Your job is not to write application features. Your job is to package, wire, harden, publish and troubleshoot the platform using Docker and Docker Compose.**

Business rules the platform must respect

- A flash sale opens with a fixed stock (for example 100 tickets). The stock counter lives in Redis so that decrements are atomic and fast.
- A buyer's request is accepted instantly (HTTP 202) and placed on a queue. A separate worker confirms the order and writes it permanently to MongoDB.
- The public site must be reachable on a single port. Neither Redis nor MongoDB may be reachable from the host machine.
- Containers must not run as root, and no secret may be baked into an image.

2. Architecture and learning outcomes

The platform has five services. Only the gateway is published to the host; everything else lives on private networks.

Service	Technology	Role	Published port	Network
gateway	Nginx (alpine)	Serves the static site and proxies /api to the API	8080 to 80	public, backend
api	Python 3.12 + FastAPI + Uvicorn	Accepts purchase requests, reads stock, exposes /health	none	backend

Service	Technology	Role	Published port	Network
worker	Python 3.12 (same codebase, different command)	Pops orders from the Redis queue, saves to MongoDB	none	backend, data
cache	Redis 7	Stock counter and order queue	none	backend, data
store	MongoDB 7	Permanent order records	none	data

Request flow: browser, then gateway, then api, which runs an atomic DECR on the stock counter in Redis and pushes an order onto a Redis list. The worker pops the order and inserts it into MongoDB.

What is different from the class demo: five services instead of three, an asynchronous queue, two databases, two custom networks, a reverse proxy instead of a dev server, a multi-stage build, non-root containers, health checks on every service, separate dev and production Compose files, and secrets supplied from files.

After this assignment you will be able to:

- Write a multi-stage Dockerfile and explain why the image gets smaller and safer.
- Run one codebase as two services using different start commands.
- Segment services with multiple networks so that data stores are unreachable from outside.
- Use health checks, depends_on conditions and restart policies to control start order and recovery.
- Layer a Compose override file on top of a base file for dev and production.
- Inject secrets with Compose secrets or env files instead of baking them into images.
- Publish versioned images to a registry and deploy from the registry only.
- Diagnose failures using logs, exec, inspect and network commands.

3. Starter code and required folder structure

Students receive the application code only (a zip or Git repository prepared by the instructor). They create everything marked NEW.

```
ticketrush/
  .env.example            NEW  non-secret settings (tag, ports)
  secrets/                NEW  one file per secret (git-ignored)
    mongo_password.txt
    redis_password.txt
  docker-compose.yml      NEW  base file, production-like
  docker-compose.dev.yml  NEW  override for development
  api/
    app/main.py           given  FastAPI app
    app/worker.py         given  queue consumer
    requirements.txt      given
    Dockerfile            NEW
    .dockerignore         NEW
  web/
    index.html            given  static page with Buy button
    nginx.conf            NEW  reverse proxy config
    Dockerfile            NEW
  README.md               NEW  run, test and troubleshoot notes
```

Contract of the given code (students must read it, not edit it)

Item	Value
API start command	uvicorn app.main:app --host 0.0.0.0 --port 8000
Worker start command	python -m app.worker
API endpoints	GET /health, GET /api/stock, POST /api/buy (JSON: name, email)
Redis settings read from environment	REDIS_HOST, REDIS_PASSWORD_FILE
MongoDB settings read from environment	MONGO_HOST, MONGO_USER, MONGO_PASSWORD_FILE, MONGO_DB
Initial stock	STOCK_TOTAL environment variable (default 100)
Password files	The code reads the password from the file path given in the *_PASSWORD_FILE variable

The app reads passwords from files on purpose, so students must mount secrets at the path they set in the variable (for example /run/secrets/redis_password).

4. Tasks (100 marks)

Task 1: Containerize the API and worker (20 marks)

- Write api/Dockerfile as a **multi-stage build**: a builder stage that installs dependencies into a virtual environment (or wheels), and a slim runtime stage (python:3.12-slim) that copies only the result.
- Create a non-root user and switch to it with USER. The container must not run as UID 0.
- Set the default CMD to start Uvicorn. Do not create a second Dockerfile for the worker.
- Write api/.dockerignore so that **pycache**, .git, .venv and test files are excluded.
- Build as <yourhub>/ticketrush-api:1.0 and record the image size.

Deliverable: Dockerfile, build output screenshot, and the output of docker run --rm <image> id showing a non-root user.

Task 2: Containerize the gateway (10 marks)

- Write web/nginx.conf: serve /usr/share/nginx/html for /, and proxy_pass /api/ to http://api:8000.
- Write web/Dockerfile from nginx:1.27-alpine that copies index.html and nginx.conf.
- Build as <yourhub>/ticketrush-web:1.0.

Deliverable: both files and one sentence explaining why the proxy target is api and not localhost.

Task 3: Compose the full stack with network segmentation (25 marks)

Write docker-compose.yml with all five services, using the images from Tasks 1 and 2 for api, worker and gateway, and official images for Redis and MongoDB.

- Create two networks: **backend** (gateway, api, worker, cache) and **data** (worker, cache, store). The api must not be able to reach MongoDB.
- Only the gateway has a ports entry. Verify by running docker compose ps and by trying to connect to Redis from the host.
- Add a named volume for MongoDB data and a named volume for Redis (append-only mode enabled with --appendonly yes).
- The worker service reuses the api image but sets command: python -m app.worker.
- Set restart: unless-stopped on every service.

Deliverable: docker-compose.yml, output of docker compose ps, and a table listing which service can reach which (use docker compose exec ... to test, for example with a short Python or nc command).

Task 4: Health checks and start order (15 marks)

- Add a healthcheck to each service: redis-cli ping for Redis (with password), mongosh ping for MongoDB, an HTTP call to /health for the api, and a check on the worker that proves its process is alive.
- Use depends_on with condition: service_healthy so the order is store and cache, then api and worker, then gateway.
- Set start_period on the slower checks and explain your timing values.

Deliverable: updated Compose file and a screenshot of docker compose ps showing every service as healthy.

Task 5: Secrets and configuration (10 marks)

- Create the files in secrets/ and declare them under the top-level secrets key. Mount them into the containers that need them.
- Pass non-secret values (image tag, public port, STOCK_TOTAL) from .env.
- Prove that no password appears in docker history <image>, in docker inspect of the image, or in the Compose file.
- Add secrets/ and .env to .gitignore.

Deliverable: the commands and outputs that prove it, plus one paragraph on why environment variables are a weaker place for secrets than files.

Task 6: Development override and resource limits (10 marks)

- Write docker-compose.dev.yml that, when layered with the base file, builds the api from source (build: ./api), bind-mounts ./api/app for live code, runs Uvicorn with --reload, and publishes the api on 8000 for debugging.
- In the **base** file, add CPU and memory limits to api and worker (for example 0.5 CPU and 256 MB) and a memory limit for MongoDB.
- Show both start-up commands: docker compose up -d and docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d.

Deliverable: the override file, output of docker compose config for the dev run, and docker stats output showing the limits applied.

Task 7: Publish and deploy from the registry (10 marks)

- Tag and push both images to Docker Hub with 1.0 and latest.
- On a clean state (docker compose down -v, then docker rmi for your images) run docker compose pull and docker compose up -d, using the base file only.

- Load-test the stock logic: send 150 purchase requests (a simple shell loop or a short script) against a stock of 100 and show that exactly 100 orders are accepted and the stock never goes negative.
- Confirm in MongoDB (via docker compose exec) that 100 order documents exist.

Deliverable: Docker Hub repository links, pull output, the load-test command with its result, and the MongoDB count.

5. Submission, evaluation and viva

Submission checklist

☐ Project folder as a zip (without secrets/ contents and without .env)
☐ README.md with run, test and troubleshooting steps
☐ Screenshots or terminal logs named task1 to task7
☐ Docker Hub links for ticketrush-api and ticketrush-web
☐ Incident reports (if attempting the challenges)

Rubric

Criterion	Marks	Full marks when
Dockerfiles	30	Multi-stage, non-root, cached layers, .dockerignore present, images build cleanly
Compose design	25	Correct networks, one published port, volumes, restart policy, worker reuses image
Reliability	15	Every health check works and start order is enforced
Security	10	No secret in image, history or Compose file; secrets git-ignored
Registry deployment	10	Fresh pull works; load test shows exactly 100 orders
Documentation	10	README lets a stranger run the stack without help
Bonus incidents	20	Clear root-cause reasoning and prevention for each fault

Viva questions (for interviews)

1. Why does the worker use the same image as the api but a different command? What would you change if their dependencies diverged?

2. What does a multi-stage build remove from the final image? Show the size difference.
3. Why is depends_on without a condition not enough for the api and Redis?
4. What would happen if the api and MongoDB shared a network? Which security principle does the data network enforce?
5. Why is the stock decrement done in Redis and not in MongoDB? What goes wrong if two requests read the stock and then write it back?
6. What is the difference between docker compose down and down -v for this project?
7. A teammate changes api code and runs docker compose up -d, but nothing changes. Give two possible reasons.
8. How would you scale the worker to three copies, and what must be true about the queue for that to be safe?

Stretch goals for strong students

- Add a GitHub Actions workflow that builds and pushes both images to Docker Hub on every push to the main branch.
- Scan the images with Docker Scout or Trivy and report the vulnerabilities found.
- Scale the worker with docker compose up --scale worker=3.

6. Incident challenges (interview practice)

This section is for interview practice, not for grading. After the stack works, use the table below as your list of faults. Inject them yourself, one at a time, by changing the configuration the way the table describes. Do not look at the fix in advance. Start from the visible symptom, then work backwards to find the cause. For each fault, explain out loud or write a short answer covering four parts: **symptom, commands used to investigate, root cause, fix and prevention.** Restore the working setup before you inject the next fault. Interviewers often ask exactly this kind of "it was working yesterday, now it is broken" question.

#	Fault injected	Visible symptom	Skills tested
1	Wrong Redis password in the secret file	api health check fails, gateway never starts	docker compose logs, healthcheck status, depends_on
2	api placed on the data network by mistake	Security requirement broken, api can reach MongoDB	docker network inspect, exec connectivity test
3	Nginx proxy_pass points to localhost:8000	Page loads, Buy button returns 502	Container networking, service-name DNS

#	Fault injected	Visible symptom	Skills tested
4	Worker container stopped	Orders accepted but MongoDB count stays at zero	Queue length in Redis, restart policy, logs
5	Memory limit of 32 MB on api	Container restarts repeatedly during load test	docker inspect (OOMKilled), docker stats
6	Image rebuilt but Compose still runs the old one	Code change has no effect	Pull vs build, image IDs, force-recreate
7	MongoDB volume removed with down -v	All orders lost after restart	Volume lifecycle, backup idea